Parallel Distributed Viewshed Analysis

J. Andrew Ware
University of Glamorgan
Division of Mathematics & Computing
Pontypridd, Wales, U.K. CF37 1DL
Tel: ++44-1443-482650
e-mail: jaware@glam.ac.uk

David B. Kidner
University of Glamorgan
School of Computing
Pontypridd, Wales, U.K. CF37 1DL
Tel: ++44-1443-482647
e-mail: dbkidner@glam.ac.uk

Philip J. Rallings
University of Glamorgan
Division of Mathematics & Computing
Pontypridd, Wales, U.K. CF37 1DL
Tel: ++44-1443-482266
e-mail: pjrallin@glam.ac.uk

1. ABSTRACT

The paper describes a number of distributed approaches to implementing a parallel visibility algorithm for viewshed analysis. The problem can be simplified by considering a range of domain partitioning strategies for optimizing the processor workloads. The best approaches are shown to work 22 times faster across a network of 24 processors. Such strategies allow traditional GIS functionality to be extended into new problem areas or to higher resolution spatial data using existing computing resources.

Keywords Intervisibility and viewshed analysis, digital terrain modeling, DEM, parallel computing, distributed computing

2. INTRODUCTION

Intervisibility analysis is a long established function of many geographical information systems [6]. In the simplest case, it is used to determine whether a target location can be seen from an observer location, termed the *line-of-sight* or *LOS*. This can be extended to consider the visible region or *viewshed* of an observer location, determined as a series of intervisibility calculations to all vertices of the digital terrain model within a pre-defined zone of visual influence. Alternatively, the viewshed can be thought of as the regions from which the target location can be seen. This function has proved popular amongst planners and landscape architects seeking to determine the visual impact of new developments. For multiple target locations, such as a wind farm, a *cumulative viewshed* will identify the prominent and sensitive regions for the planners. This in turn can be extended to consider the *reverse viewshed* or *visibility indices* of a region. The visibility index of a location is defined as the number of entities that are visible from that point. When calculated for an extensive region covering a proposed development, this reverse viewshed gives an indication of the site areas sensitive to visual impact, which can be taken into account during planning. In relation to our example of a wind farm development, a cumulative viewshed might be calculated for 20 pre-defined turbine locations, whilst a reverse viewshed might be calculated for thousands of possible locations within the site (Figure 1).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
ACM GIS '98 11/98 Washington, D.C., USA
© 1998 ACM 1-58113-115-1/98/0011...$5.00

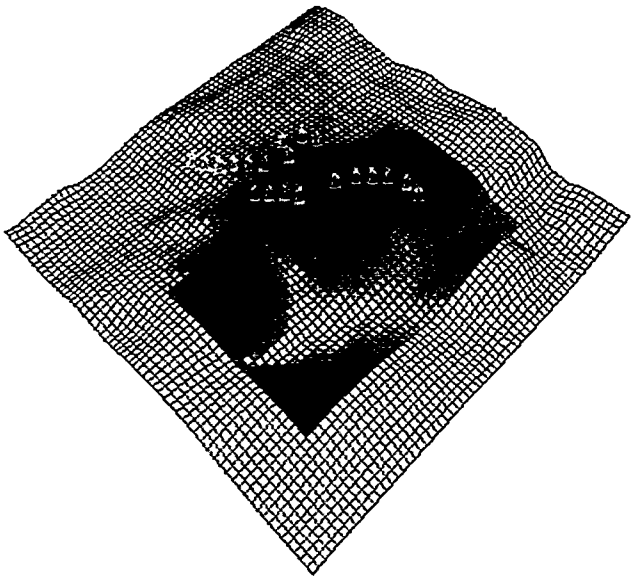


Figure 1. The reverse viewshed for a 2 km x 2 km (or 41 x 41 vertices) wind farm in South Wales. The visibility index for each vertex is calculated as the number of LOS profiles within a 20 km radius. Darker regions indicate areas of higher visibility. The actual wind turbine locations are also shown.

This extension of the viewshed problem inevitably means that uni-processor systems provide poor response times when calculating the visibility indices of more than just a few locations, as may be required for environmental impact assessment and site selection [10]. In these circumstances parallel processing techniques can be applied to computationally intensive GIS applications.

Intervisibility analysis is already an established application area for parallel processing [5, 11, 12, 16]. More often though, research has focused on using fine grain

architectures or Transputer networks. This is indicative of much of the published literature that addresses parallel GIS applications. Today, the increasing availability of computer networks, combined with recent advances in modern PC operating systems means that many organizations already have an existing multi-purpose parallel processing resource which could be utilized to carry out such tasks.

This paper discusses the issues involved when implementing this reverse viewshed function on a distributed cluster of networked machines. As vast quantities of spatial data become available, particularly DEMs at larger scales and denser resolution, the demands for parallel processing will inevitably increase.

3. DIGITAL ELEVATION MODELS

The most common approach to representing the terrain surface is as a regular grid digital elevation model or DEM. A DEM records the elevation of the surface at fixed intervals of x and y, such as every 10 meters for Ordnance Survey (O.S.) 1:10,000 scale data, 50 meters for O.S. 1: 50,000 scale data, or 30 meters for United States Geological Survey (U.S.G.S.) 1: 24,000 scale quadrangles. The advantages of using such a data structure for representing the terrain is that the uniform grid minimizes search operations, since the co-ordinates of an elevation are implicitly related to the matrix position. Algorithms are also conceptually very simple when applied to the regular grid.

It is apparent that the processing workload will be dependent upon the geographical extent of the area under consideration and the resolution of the terrain database. It is common for an Environmental Impact Statement of a proposed development to consider the viewshed over an area extending up to 20km from the site.

In the site selection process, the locations which are chosen for calculating the visibility indices are not predetermined. The simplest scenario is to calculate the visibility index for each DEM elevation, or alternatively at fixed intervals at the sub-cell level within the site. In practice, however, typical visibility indices would be quoted in terms of tens of thousands of DEM vertices, and our indices calculated for hundreds or thousands of possible locations. To illustrate the workload involved, if the visibility indices for each vertex in a 20 x 20 kilometer DEM are calculated as the cumulative line-of-sight to every other point in the DEM, the necessary processing time required would be 299 days at a resolution of 50 meters (1:50,000) or 508 years at a resolution of 10 meters (1:10,000). None of these estimated timings take into account the larger number of interpolations required at the 10 meter resolution, so these estimates of one millisecond per profile would also increase. These timing estimates also assume the algorithm uses bilinear interpolation rather than one of the smoother sampling strategies such as bicubic interpolation. Similarly, the time for each LOS calculation increases significantly when we consider surface features on the landscape, such as buildings or vegetation, since point-in-polygon or line intersection checks need to be made [6].

This illustrates the need for a parallel algorithm to address this type of application. Whilst parallel processing will make some inroads into the development of a more efficient approach to calculating DEM visibility indices and reverse viewsheds, the problem may also have to be simplified in order to make this type of analysis viable for inclusion within the broader confines of a large scale site selection GIS. This could simply include reducing the number of observation or target points, or even using a multi-scale approach which begins at a coarse resolution and is refined at higher resolutions where there appear to be sharp changes in visibility.

The solution to obtaining increased hardware performance is theoretically quite simple - either design a faster central processing unit or use more of them [15]. Despite the perpetual upgrading of computer technology and design, the demands of some of these GIS applications have grown at an increased rate. In these instances, we need to consider the alternative option of using multiple processors, either within the same hardware platform or across a distributed network.

4. PARALLEL WORKSTATION CLUSTER

Advances in the fields of networks and operating systems have provided organizations with a valuable non-specialized, general-purpose parallel processing resource. Networks are commonplace, and when combined with one of the many fully multi-tasking and pre-emptive operating systems, it is possible to build a parallel processing cluster of machines. Operating systems which are able to support these environments include Windows NT, OS/2, LINUX, Solaris, and to a limited extent, Windows 95. These parallel processing clusters can be homogeneous (same machine architecture) or heterogeneous in nature and can link machines or workstations of various speeds and abilities. Each machine in a cluster can be a complete system in its own right, usable for a wide range of other computing tasks. Specialized hardware (parallel or otherwise) can also be incorporated into the cluster or virtual machine.

The main advantage of using a cluster is that many organizations have a network of fast processor machines (e.g. Pentium PC-based machines). Whilst each single system cannot on its own perform the whole task, they are able to process part of the overall job. Cluster computing can scale to provide a very large parallel machine and specialized hardware can be made available to all machines (e.g. a machine in the cluster could forward data to an image processing workstation so that the data can be mapped in to a visual form). Each individual machine would also have total and independent control of its own resources (e.g. memory, disk, etc.) which can also be

exploited (e.g. a machine typically has more memory than processors in a more specialized computer environment). Clusters thus enable an organization to realize the full potential of a computer network, and the combined processing power can handle even the most extensive GIS application.

General-purpose network hardware is not designed for parallel processing systems, especially when masses of data have to be passed between systems. Latency is very high and bandwidth is relatively low compared to specialized parallel processing environments. However, recent developments in network technology make it possible for workstations to have either a higher shared or a dedicated bandwidth (e.g. the cluster shares a 100 mbs bandwidth or each machine has its own dedicated 10 mbs bandwidth). Message passing and communication is the main overhead when using a parallel cluster and is an issue that must be addressed when implemented into GIS applications. Furthermore, if a heterogeneous network is created, some message encoding system must be incorporated so that all machines can communicate effectively, regardless of processor architecture or operating system.

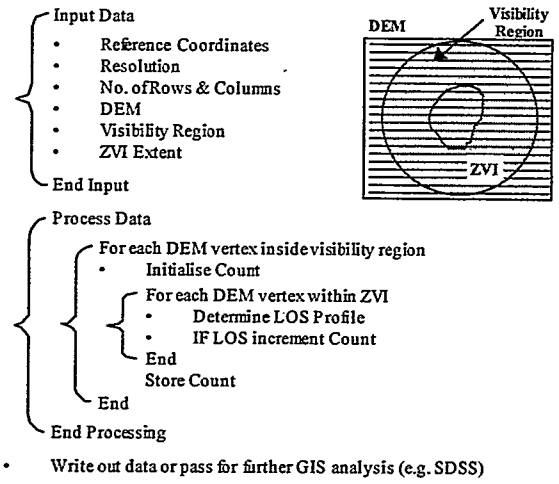
There are a number of readily available software packages for assisting the implementation of distributed systems. These include PVM [8], Express [7], MPI [14], and Linda [3]. Various other systems with similar capabilities are also in existence; Turcotte [17] provides an exhaustive list and overview. To be of practical use, the software must support a number of different vendors and architectures (unless of course, only a homogenous cluster is created).

5. REVERSE VIEWSHED ALGORITHM

Parallel processing involves either splitting the process into several sub-processes and performing them on different processors concurrently, or splitting the data that is to be processed between a number of processors and executing multiple copies of the process simultaneously. Healey & Desa [9] categorize algorithm parallelism as being one of three broad approaches, namely event, geometric and algorithmic parallelism. Event parallelism, or farming, utilizes the concept of a master processor distributing tasks to slave processors and assembles the computational results from each in turn. Geometric, or domain parallelism partitions the data set over the available processors, whilst algorithmic, or function parallelism partitions the process into sub-tasks.

In the case of the visibility algorithm, the number of distinct tasks is relatively few (Figure 2). The processing overheads are due to the sheer quantity of possible LOS profiles to be analyzed. Calculation of the profile is relatively straightforward, depending on the algorithm used and the type of interpolation employed. For the purpose of calculating visibility indices, linear or bilinear interpolation is generally considered sufficient. The algorithm of Figure 2 assumes that the visibility indices are to be calculated for all DEM vertices within a defined visibility region (polygon) to all other DEM vertices within a zone of visual influence (ZVI).

Figure 2. Serial algorithm for determining the visibility index or reverse viewshed.



The main implementation issue is deciding how to segment the data, such that each processor is given an equivalent amount of work, thus minimizing idleness and producing a load-balanced system. The load-balancing issue is even more exaggerated given the nature of a heterogeneous network connecting processors of different speeds and abilities. In this respect, the equivalent amount of work would be measured in time rather than volume, with quicker processors being assigned a greater amount of work. The assignment process would be influenced by some measure of machine performance when compared with others or a base machine. This would enable faster resources to be exploited to their full potential.

Another consideration of domain decomposition is overcoming or minimizing the problem of accessing data from neighboring sub-regions. Healey & Desa [9] point out that as individual sub-regions are handled by different processors, boundary data must be either duplicated (overlap) or passed between processors. It is therefore important to define sub-regions, such that the amount of within region processing is maximized and the between region communication is minimized. The positioning of regions per processor is relatively unimportant in a distributed environment, unless some workstations posses special communication facilities. In the latter case, the areas most likely to generate inter-region communication would be placed on these workstations.

6. DOMAIN PARTITIONING STRATEGIES

The parallelizing of the reverse viewshed function is clearly an issue of domain decomposition, which can be achieved in two scenarios; applying all of the vertices of

the visibility region (for which visibility indices are to be calculated) to part(s) of the DEM, and applying subset(s) of the visibility region to all of the DEM.

The partitioning of raster data such as DEMs for parallel algorithms requires methods for reading data from disk, distribution of data amongst the processors, gathering the resultant data from the processors, and writing data to disk [13]. A number of alternative approaches exist, each of which imposes a different pattern of coordination and communication between processes.

The partitioning of the domain, or work allocation, can be achieved either statically or dynamically. Static allocation involves dividing the DEM between the processors' workforce at the start of the process, such that the work allocation does not alter during runtime. The scale of the domain allocated per processor is called the grain size or granularity. The work dividing algorithm would have to allocate the domain evenly amongst all available processors, but if there is no a priori knowledge of the processing load then the domain allocation would be one of dividing the workload into equal areas. Static domain decomposition requires regularity in the structure of the domain, uniformity of the workforce requirements, and uniformity of the working capabilities of the workforce. Any non-uniformity in any one of the aforementioned areas may render the decomposition of the domain in such a way as not to be able to preserve processor efficiency. Static allocation is well suited to environments with identical or equally matched processors, however careful attention must be given to the work dividing algorithm such that the work is actually divided into equal amounts of processing time. If certain areas require more processing due to terrain specific features, then the problem of load balancing becomes an even greater issue when preserving overall system efficiency.

Dynamic allocation involves allocating workload (or domain segments) to processors throughout the entire system runtime. Typically worker processors are assigned tasks by a master process, whilst upon completion, the workers return any results and are subsequently assigned more tasks. Such a process is commonly termed *farming* or collectively known as *processor farms* and is a practical way of implementing dynamic allocation. Dynamic allocation is needed when any one of the following is true: the domain structure is irregular and does not match workforce structure; work is distributed non-uniformly across the domain; and work capability is distributed non-uniformly to the workforce. Dynamic allocation is capable of supporting a diverse range of processors and is able to utilize individual capabilities and speeds. The disadvantage of farming is the management overhead, mainly associated with having dedicated control nodes. The additional dynamic management incurs extra communication that is not inherent in the problem and exists solely to support dynamic allocation of the domain. The communication overhead further increases as additional nodes are added to the processor farm which can produce a communication bottleneck at the control (or farmer) node. The only way to alleviate this is to apply recursion by introducing more control nodes to form a management hierarchy. Dynamic allocation is well suited for heterogeneous networks of varying speed processors and in applications where the workload (or domain) cannot easily be allocated according to processing time. However, the extra management overhead can be minimized to provide better utilization of processors.

One of the main concerns in domain decomposition is the granularity or size of work allocated each time, which is especially true in dynamic allocation, where the granularity can have a dramatic effect on processor efficiency and management overhead.

The visibility algorithms implemented include both static and dynamic allocation of the whole DEM and the visibility region. Dynamic allocation methods have been used with varying degrees of granularity and the resultant overheads assessed. Full results have been processed for 1 to 24 processors on 6 static and 5 dynamic (excluding granularity) allocation algorithms. The different algorithms are compared with respect to overall speed up and processor efficiency.

Two comparison indices [1] were observed for each work dividing algorithm; speed-up and processor efficiency. Speed-up relates to the time taken to compute a task on a uni-processor system to the time taken to compute the same problem using the parallel implementation. Speed up is thus defined as:

$$Speed\text{-}Up = \frac{Elapsed\ Time\ of\ a\ Single\ Processor}{Elapsed\ Time\ of\ the\ Multi\text{-}Processors}$$

The relative efficiency, based on the performance of the problem for one processor, can be a useful measure as to what percentage of a processor time is used on computation. Any implementation penalty would be immediately reflected by a decrease in the relative efficiency:

$$Efficiency = \frac{Speed\text{-}Up \times 100\%}{Number\ of\ Processors}$$

In terms of comparison, the relative efficiency indicator is the major factor since it has an implied time element in terms of implementation overheads and shorter execution. The efficiency factor also gives an indication as to the effective load balancing of the partitioning algorithm.

The main consideration in developing static algorithms is to ensure that the partitioning is efficiently load-balanced. If local processing is concentrated on a single (or even few) isolated areas of the DEM, load balancing can be compro-

mised at the expense of overall efficiency, a factor that is emphasized as more processors are added to the system. However, static algorithms are relatively simple and impose no great communication or implementation overhead.

The main area of investigation into dynamic farming algorithms is to determine the optimal granularity for a given domain and processor network. The amount of work allocated each time has a substantial effect on the overall system. A small granularity increases the number of times processors request additional work and thus increases the communication overhead, however if the granularity is larger, then the communication overhead is reduced at the expense of load balancing. The farming algorithms are implemented with varying degrees of granularity with comparisons made as to the overall effectiveness.

7. TESTS AND RESULTS

The parallel visibility algorithms were applied to the problem of determining the reverse viewshed of a 2 km x 2 km visibility region of a larger 22 km x 22 km DEM of South Wales (the ZVI). The region is the location of the Taff Ely Wind Farm, 15 kms north-west of Cardiff, and is the subject of an on-going investigation into the performance of GIS tools for the visibility analysis of Wind Farms [6]. The DEM was sampled at a resolution of 50 m, hence the indices were calculated for 1,681 points (i.e. 41 x 41 vertices) as the number of line-of-sight profiles to every other DEM vertex (194,481 points, i.e. 441 x 441 vertices). This is equivalent to nearly 327 million profile calculations; each of which could consist of up to 1,000 interpolations.

The parallel processing cluster was established for twenty-four 100 Mhz Pentium based machines, connected via a standard 10 mbs ethernet network, running Windows NT Workstation 4.0. The software was developed using the PVM (Parallel Virtual Machine) system and the C programming language. PVM can be compiled on over 40 platforms including specialized and non-specialized parallel processing hardware, including JavaPVM for distributed processing over the Internet. PVM is ideal for generating code, since applications are portable over the PVM system, regardless of machine architecture.

For the static algorithms, most notable was the apparent ineffectiveness of algorithms that center processing on one isolated area of the DEM. In terms of speed-up performance the partitioning algorithms were very similar, although a more noticeable divergence was identified as the number of processors increases. The efficiency of these static approaches is undoubtedly affected by the difficulty in load balancing the processors. Hills which block visibility will seriously influence the validity of such approaches. Algorithms that partition the workload on isolated areas proved to be inefficient, whilst algorithms that diffused processing completely over the entire domain proved to be extremely erratic in terms of efficiency. The latter subset provided some of the best static efficiency values, however these were not consistent and decreased dramatically depending on the number of processors.

The third area of static algorithms include those that center processing over a few isolated areas distributed over the entire DEM. These provided a more stable load balancing and more consistent efficiency.

Overall the dynamic partitioning approaches produced marginally better speed-up than the static algorithms, however they produced more consistent efficiency indicators and hence a more efficient load balanced system. The dynamic approach produced efficiency factors of over 100% compared to the uni-processor system, this is in part, due to the different structure of the farming program. In view of this it may be possible to restructure the uni-processor system to achieve a better performance, such that the results could then be compared. The results demonstrated the ineffectiveness of farming with extremes of granularity (too high or low), suggesting a need to observe the optimal granularity for dynamic farming algorithms. From the results it can be deduced that the optimal efficiency is produced when the granularity represents approximately 0.1 - 0.3 % of the domain, since the efficiency drops when the granularity represents 0.5 - 1% and is totally inefficient when the granularity is over 5%. Inefficient results are also observed when the granularity is less then 0.05% of the domain.

The communication overhead involved in managing dynamic farming was quite low, thus demonstrating the effectiveness of the implementations. However the overhead increases as processors are added, so at some point it will become inefficient, although the results are promising even on 24 processors. The communication overhead [2] is measured as a percentage of the overall parallel implementation time (not the uni-processor time) and is a necessary burden to ensure that the system is load balanced, with the main factor being the granularity of work allocated. Even on the inefficient algorithms the communication overhead is not excessive, proving that the major drawback is load balancing.

8. CONCLUSIONS

The results prove that load balancing is difficult with static decomposition and can be erratic in relation to the number of processors. Dynamic farming strategies, on the other hand, produced better results and are far more resilient in relation to the processor network and load balancing - this being true when there is no extreme level of granularity.

The algorithms have been tested on a network of homogeneous processors of the same speed, but the problems of implementing static algorithms would be even more pronounced when heterogeneous processors of varying speeds were added to the network. These problems

would increase depending on the varying degrees of machine capabilities. Static algorithms produce less efficiency as more processors are added, even of the same type, and would deteriorate further in a heterogeneous network.

Dynamic processing farms would not suffer from the same penalties, as every processor could be utilized due to the inherent load balancing properties of farming. However, it may be the case that in heterogeneous networks, the granularity may prove to be a determining factor than in homogeneous networks, where there is a liberal freedom in terms of granularity.

Currently only a one tier system has been investigated in which one master node communicates with all processing nodes in a tree or hierarchical fashion, and all sub-processes are created and communicate with the master process. In addition, it is currently assumed that all processors have access to the entire DEM. The multi-layered model [4] would allow the DEM to be divided between additional control nodes such that each sub-system would reflect the overall system in that it would apply the work dividing algorithms to its own local domain, a sub-domain of the overall DEM. This model would be capable of processing much larger DEMs, at larger resolutions and still present a viable, well structured distributed GIS.

The additional control nodes would certainly be beneficial to the dynamic farming algorithms, even if each processor continued to have access to the entire DEM. Our research suggests that whilst one master node is sufficient for 24 processors the results indicate that the management communication overhead would spiral as extra processors are added. It is doubtful that a single tree system could cope with an additional 10 nodes when implementing the dynamic algorithms.

Parallel processing of GIS problems is still in its infancy, despite the steady stream of academic conference papers which have addressed the issues over the last 10 to 15 years. The need for specialized hardware was certainly a major obstacle to its acceptance by a wider community outside of academia. However, as most organizations now use networked machines, parallel processing has the potential to benefit a much wider audience. The majority of applications that could benefit from parallelization, such as visibility analysis, are conceptually very simple. The real problem is identifying the best strategy for assigning the workloads to machines which minimizes both processor redundancy and communication overheads. This paper has shown that this visibility problem can be solved 22 times faster with 24 processors. It remains to be seen whether the GIS community will adopt parallel computing, but the technology and hardware is now in our reach.

9. REFERENCES

[1] Agrawal, D.P. and Pathak,G.C. Evaluating the performance of microcomputer configurations. IEEE Computer, 19:23-27, 1986.

[2] Annaratone, M., Pommerell, C., and Ruhl, R. Interprocessor communication speed and performance in distributed-memory parallel processors. In 16th Symp. on Computer Architectures, pp. 315-324, 1989.

[3] Carriero, N. and Gelernter, D. LINDA in Context, Commun. ACM, Vol 32, April 1989, pp. 444-458.

[4] Dandamundi, S.P. and Eager, D.L. Hierarchical interconnection networks for multicomputer systems. IEEE Trans. Computers, 29(6) pp. 786-797, 1990.

[5] De Floriani, L., Montani, C. and Scopigno, R. Parallelizing Visibility Computations on Triangulated Terrains, Int. J. of GIS, 8(6), pp. 515-531, 1994.

[6] Dorey, M., Sparkes, A., Kidner, D., Jones, C. & Ware, M. Terrain modelling enhancement for intervisibility analysis, in Ch. 14 of Innovations in GIS 6, Ed: B. Gittings, London: Taylor & Francis, 1998.

[7] Flower, J., Kolawa, A. and Bharadwaj, S. The express way to distributed processing, Supercomputing Review, May 1991, pp. 54-55.

[8] Geist, A., Beguelin, A., Dongarra, J., Jiang W., Manchek, R. and Sunderam, V. PVM: Parallel Virtual Machine - a user's guide and tutorial for networked parallel computing, 1994.

[9] Healey, R.G. & Desa, G.B. Transputer-based parallel processing for GIS analysis: problems and potentialities, Proc. of Auto-Carto 9, ACSM/ASPRS, Baltimore, April 1989, pp. 90-99.

[10] Kidner, D.B., Sparkes, A.J. and Dorey, M.I., Geographical Information Systems in Wind Farm Planning, in Ch. 9 of *Geographical Information and Planning: European Perspectives*, (Eds: S.Geertman, S.Openshaw, & J. Stillwell), Springer-Verlag, 1998.

[11] Kidner, D.B., Rallings, P.J. and Ware, J.A. Parallel processing for terrain analysis in GIS: visibility as a case study, GeoInformatica, 1(2), 1997, pp. 183-207.

[12] Mills, K., Fox, G., & Heimbach, R. Implementing an Intervisibility Analysis Model on a Parallel Computing System, Comp. & Geosciences, 18(8), 1992, pp.1047-1054.

[13] Mineter, M.J. Partitioning raster data, in Ch. 10 of Parallel Processing Algorithms for GIS, Editors: R. Healey, S. Dowers, B. Gittings & M. Mineter, London: Taylor & Francis, pp. 215-230, 1998.

[14] MPI Forum MPI: A Message-Passing Interface Standard, Int. J. of Supercomputing Applications, 8(3/4), 1994.

[15] Strand, E.J. Multiple processors promise powerful GIS advancements, GIS Europe, May 1992, pp. 34-36.

[16] Teng, Y.A., DeMenthon, D., & Davis, L.S. Region-to-region visibility analysis using data parallel machines, Concurrency: Practice & Experience, 5(5), 1993, pp. 379-406.

[17] Turcotte, L. A survey of software environments for exploiting networked computing resources, Technical Report, Mississippi State University, 1993.